



System Modeling

CHAPTER 5-SOFTWARE ENGINEERING BY IAN
SOMMERVILLE

Content

- ▶ Context models
- ▶ Structural models
- ▶ Behavioral models

System Modeling

- ▶ System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- ▶ System modeling has generally come to mean representing the system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML)

System Model

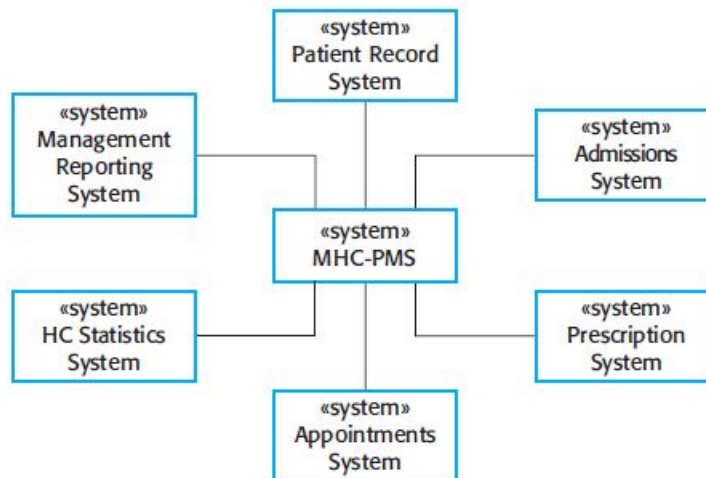
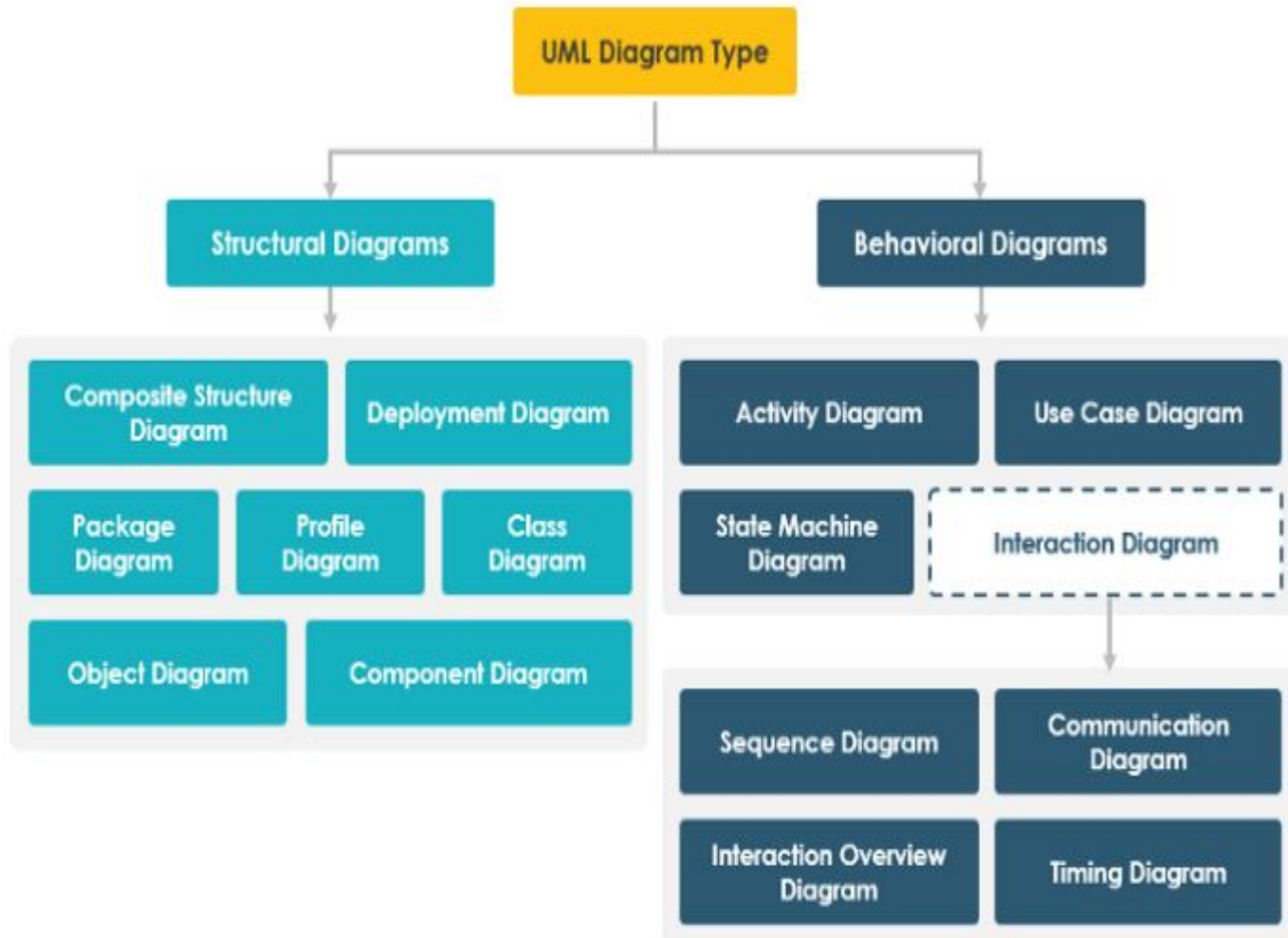


Figure 5.1 The context of the MHC-PMS

Static vs Dynamic Modeling

- ▶ A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- ▶ A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events.



Static vs Dynamic Modeling

UML diagrams represent these two aspects of a system:

- ▶ **Structural (or Static) view:** emphasizes the static structure of the system using objects, attributes, operations and relationships. It includes class diagrams and composite structure diagrams.
- ▶ **Behavioral (or Dynamic) view:** emphasizes the dynamic behavior of the system by showing collaborations among objects and changes to the internal states of objects. This view includes sequence diagrams, activity diagrams, and state machine diagrams.

The Unified Modeling Language

- ▶ The Unified Modeling Language is a set of 13 different diagram types that may be used to model software systems.
- ▶ It emerged from work in the 1990s on object-oriented modeling where similar object-oriented notations were integrated to create the UML.
- ▶ A major revision (UML 2) was finalized in 2004.
- ▶ The UML is universally accepted as the standard approach for developing models of software systems.
- ▶ Variants have been proposed for more general system modeling.

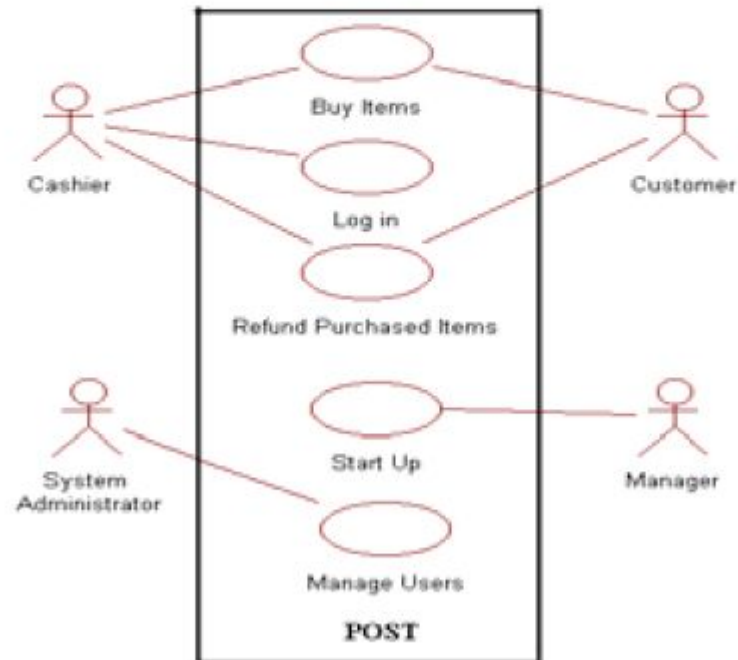
Use-case Diagram

- ▶ A use case diagram is a graphical depiction of a user's possible interactions with a system.
- ▶ A use case diagram shows various use cases and different types of users the system has and will often be accompanied by other types of diagrams as well



Sample Solution for POST..Continued

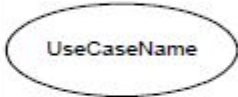
Example - Use Cases Diagram



MHC-PMS: Transfer data

Actors	Medical receptionist, patient records system (PRS)
Description	A receptionist may transfer data from the MHC-PMS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary
Stimulus	User command issued by medical receptionist
Response	Confirmation that PRS has been updated
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

Use Case Modeling: Core Elements

Construct	Description	Syntax
use case	A sequence of actions, including variants, that a system (or other entity) can perform, interacting with actors of the system.	
actor	A coherent set of roles that users of use cases play when interacting with these use cases.	
system boundary	Represents the boundary between the physical system and the actors who interact with the physical system.	

Use Case Modeling: Core Elements

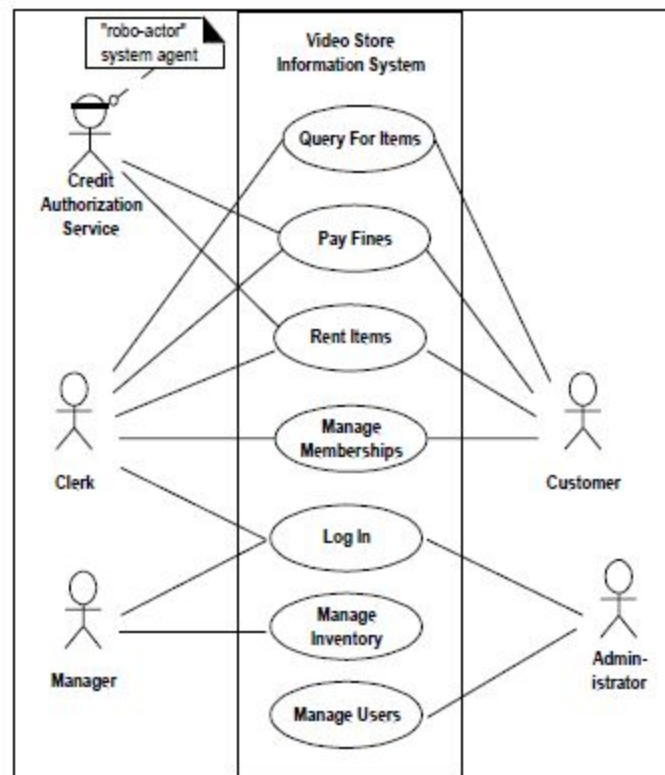
Construct	Description	Syntax
association	The participation of an actor in a use case. i.e., instance of an actor and instances of a use case communicate with each other.	_____
generalization	A taxonomic relationship between a more general use case and a more specific use case.	—————>
extend	A relationship from an <i>extension</i> use case to a <i>base</i> use case, specifying how the behavior for the extension use case can be inserted into the behavior defined for the base use case.	<<extend>> ----->

Use Case Modeling: Core Elements

Construct	Description	Syntax
include	An relationship from a <i>base</i> use case to an <i>inclusion</i> use case, specifying how the behavior for the inclusion use case is inserted into the behavior defined for the base use case.	<<include>> ----->

Use Case Diagram

- A way to conceive and illustrate the use cases.
- Usually created during the initial use case analysis.



Use-case & Dynamicity

- ▶ To model a system, the most important aspect is to capture the dynamic behavior. Dynamic behavior means the behavior of the system when it is running/operating.
- ▶ Only static behavior is not sufficient to model a system rather dynamic behavior is more important than static behavior. In UML, there are five diagrams available to model the dynamic nature and use case diagram is one of them. Now as we have to discuss that the use case diagram is dynamic in nature, there should be some internal or external factors for making the interaction.

Purpose of Use Case Diagrams

- ▶ The purpose of use case diagram is to capture the dynamic aspect of a system

Purpose of Use-case Diagram

In brief, the purposes of use case diagrams can be said to be as follows –

- ▶ Used to gather the requirements of a system.
- ▶ Used to get an outside view of a system.
- ▶ Identify the external and internal factors influencing the system.
- ▶ Show the interaction among the requirements and actors.

How to Draw a Use Case Diagram?

- ▶ When we are planning to draw a use case diagram, we should have the following items identified.
 1. Functionalities to be represented as use case
 2. Actors
 3. Relationships among the use cases and actors.

Cont..

- ▶ The name of a use case is very important. The name should be chosen in such a way so that it can identify the functionalities performed.
- ▶ Give a suitable name for actors.
- ▶ Show relationships and dependencies clearly in the diagram.
- ▶ Do not try to include all types of relationships, as the main purpose of the diagram is to identify the requirements.
- ▶ Use notes whenever required to clarify some important points.

Example :Order Management System

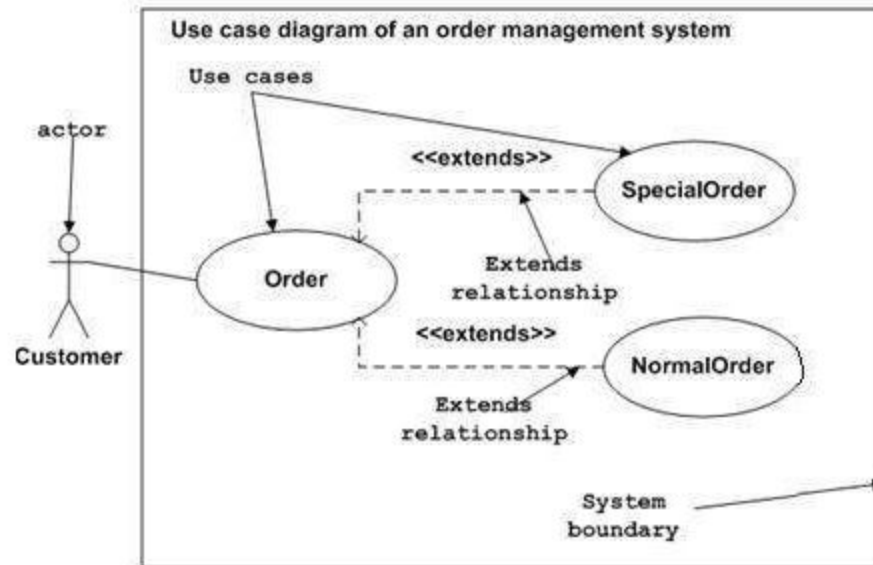


Figure: Sample Use Case diagram

Example: Explanation

The SpecialOrder and NormalOrder use cases are extended from *Order* use case. Hence, they have extended relationship. Another important point is to identify the system boundary, which is shown in the picture. The actor Customer lies outside the system as it is an external user of the system.

Usecase Relationships

There can be 5 relationship types in a use case diagram.

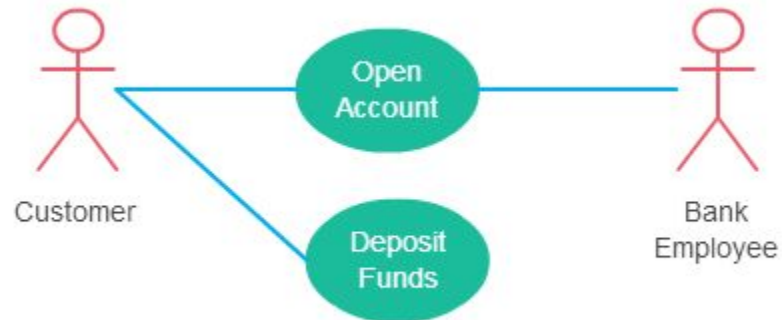
- ▶ Association between actor and use case
- ▶ Generalization of an actor
- ▶ Extend between two use cases
- ▶ Include between two use cases
- ▶ Generalization of a use case

Association Between Actor and Use Case

This one is straightforward and present in every use case diagram.
Few things to note.

- ▶ An actor must be associated with at least one use case.
- ▶ An actor can be associated with multiple use cases.
- ▶ Multiple actors can be associated with a single use case

Example

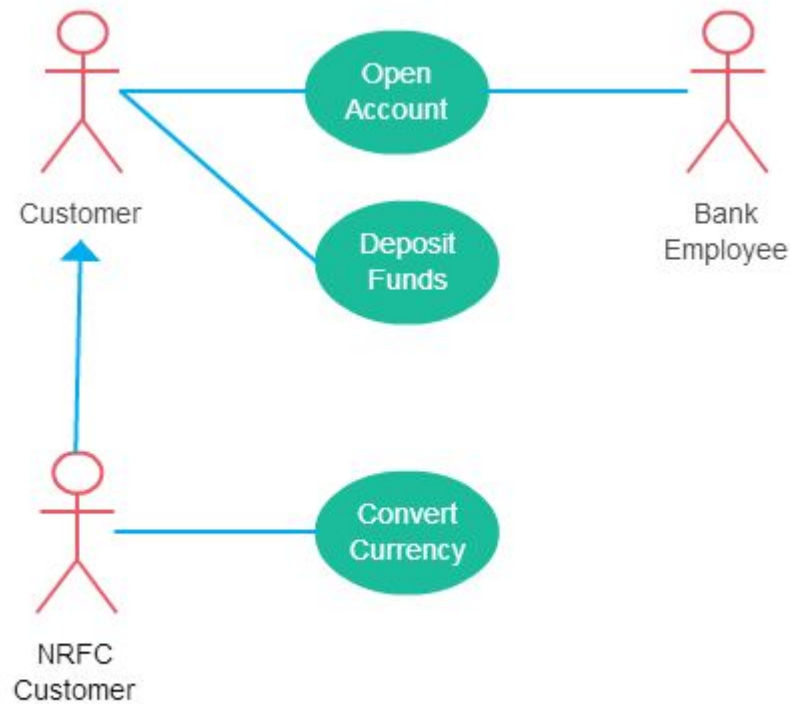


Different ways association relationship appears in use case diagrams

Generalization of an Actor

- ▶ Generalization of an actor means that one actor can inherit the role of the other actor. The descendant inherits all the use cases of the ancestor. The descendant has one or more use cases that are specific to that role. Let's expand the previous use case diagram to show the generalization of an actor

Example

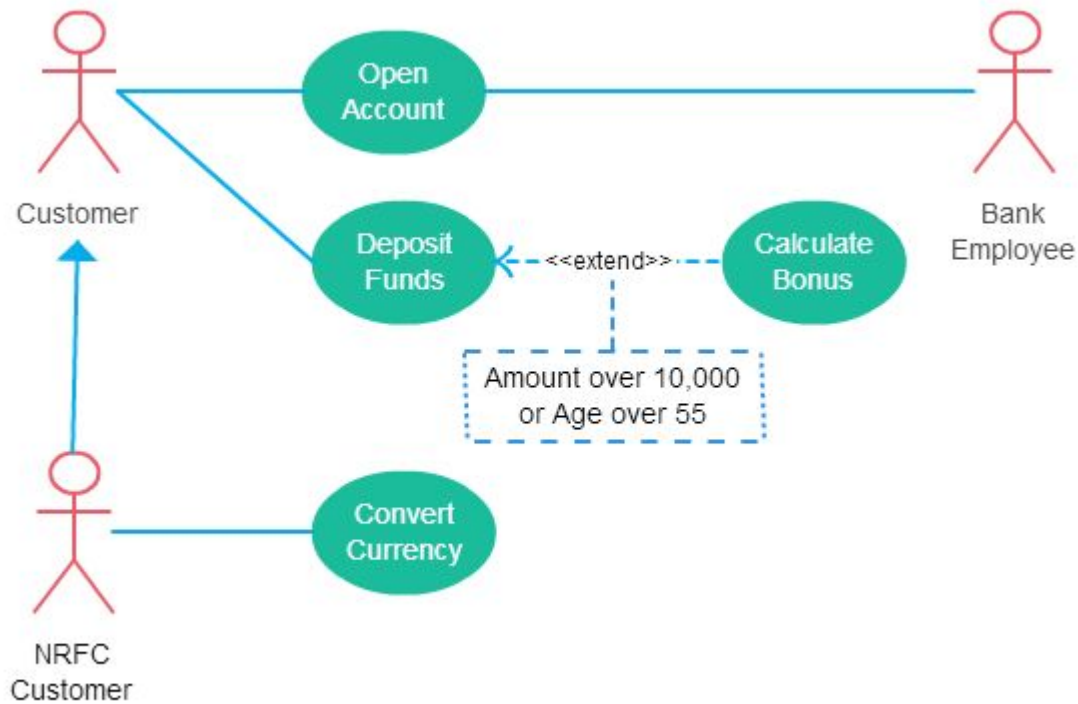


A generalized actor in an use case diagram

Extend Relationship Between Two Use Cases

- ▶ Many people confuse the extend relationship in use cases. As the name implies it extends the base use case and adds more functionality to the system. Here are a few things to consider when using the <<extend>> relationship.
- ▶ **The extending use case is dependent on the extended (base) use case.** In the below diagram the “Calculate Bonus” use case doesn’t make much sense without the “Deposit Funds” use case.
- ▶ **The extending use case is usually optional** and can be triggered conditionally. In the diagram, you can see that the extending use case is triggered only for deposits over 10,000 or when the age is over 55.
- ▶ **The extended (base) use case must be meaningful on its own.** This means it should be independent and must not rely on the behavior of the extending use case.

Example



Extend relationship in use case diagrams

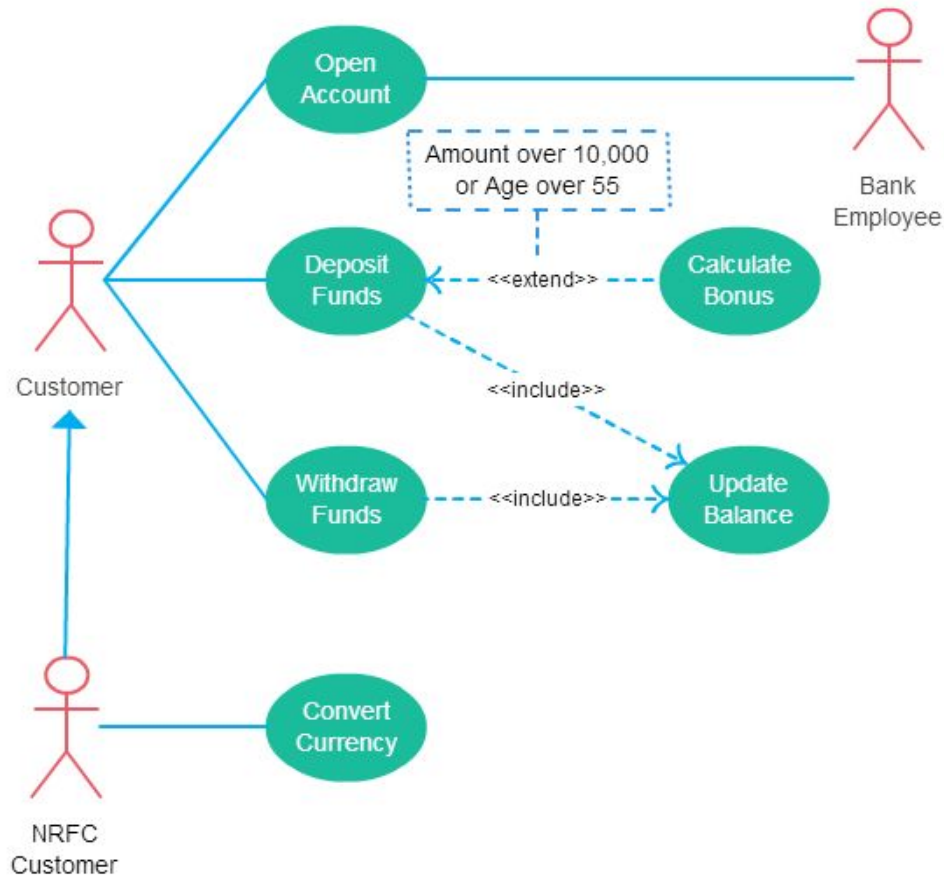
Example

- ▶ Although extending use case is optional most of the time it is not a must. An extending use case can have non-optional behavior as well. This mostly happens when your modeling complex behaviors.
- ▶ For example, in an accounting system, one use case might be “Add Account Ledger Entry”. This might have extending use cases “Add Tax Ledger Entry” and “Add Payment Ledger Entry”. These are not optional but depend on the account ledger entry. Also, they have their own specific behavior to be modeled as a separate use case.

Include Relationship Between Two Use Cases

- ▶ Include relationship show that the behavior of the included use case is part of the including (base) use case. The main reason for this is to reuse common actions across multiple use cases. In some situations, this is done to simplify complex behaviors. Few things to consider when using the <<include>> relationship.
- ▶ The base use case is incomplete without the included use case.
- ▶ The included use case is mandatory and not optional.

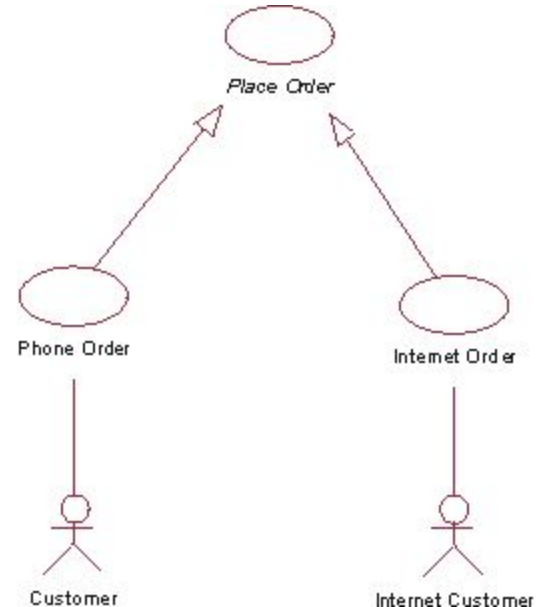
Example



Includes is usually used to model common behavior

Generalization of a Use Case

- ▶ This is similar to the generalization of an actor. The behavior of the ancestor is inherited by the descendant. This is used when there is common behavior between two use cases and also specialized behavior specific to each use case.



Example

For example, in the previous banking example, there might be a use case called “Pay Bills”. This can be generalized to “Pay by Credit Card”, “Pay by Bank Balance” etc.

Descriptive Usecase

- ▶ A use case description is a text-based narrative of a functionality comprised of detailed, step-by-step interaction between the actor and the system.
- ▶ It describes the outcomes of an action taken to accomplish a specific goal.
- ▶ It also details different paths that can be followed by defining primary, alternate, and exception flows.
- ▶ Use cases are written from the point of view of the actor and avoid describing the internal aspects of the solution.

Descriptive Use-case Template

Use Case Template

Name, description	Brief description of the process (what is happening) in the use case	
Actors	List all actors (people, systems, etc.) associated with this requirement	
Pre-condition	What must occur before the use case begins	
Post-condition	What has occurred as a result of the use case	
Main Success Path (primary flow)	Description of the sequence of activities in the most commonly completed path or flow. The main flow is the most routine path from the pre- to the post-conditions.	
Actor Actions	System Responses	
1. List the actor steps here	1.1 List the system steps here	
2.	2.1	
Alternate Path	A1	List the description here, and the less common sequences below that get to the post-conditions. Include any triggers, and where the use case resumes after this flow.
Actor Actions	System Responses	
1. List the actor steps here	1.1 List the system steps here	
2.	2.1	
Exception Path	E1	List the description here, and the sequence of actions that prevents getting to the post-conditions. Include any triggers.
Actor Actions	System Responses	
1. List the actor steps here	1.1 List the system steps here	
2.	2.1	
Scenarios (Insert additional rows for each scenario)		
Scenario	Post-Conditions	Flow

Example-Search Student

Search student
Brief description: The actor receives results of students
Actors: Participant
Preconditions: The actor is a registered student
Basic flow of events: 1. The actor logs into the system 2. The system authenticates the actor and starts a session 3. The actor chooses to search for student 4. The actor is guided by the system to fill the required criteria's to search for student 5. The actor receives results for student 6. The actor leaves the system
Extensions: 1a. The system fails to authenticate the actor. • The system informs the actor and doesn't allow the actor to proceed 3a. The system fails to search • The system informs the actor
Post-conditions: The system outputs results for the search of student
Special requirements: Actor is connected to Internet and uses a Java enabled Mobile phone